

DubzChain Phase 6 — Public Testnet Maturity Guide

A concise operator packet covering the remaining work to finish Phase 6: operator docs, launch recipes, seed-node guidance, frozen constants, and release discipline.

Phase 6 item	Status
6.1 Operator docs	Drafted
6.2 Clean launch recipes	Drafted
6.3 Seed-node deployment guidance	Drafted
6.4 Frozen network constants	Drafted
6.5 Release discipline	Drafted

6.1 Operator Docs

Network identity

- chainId: dubzchain-devnet
- protocolVersion: 1
- networkMagic: 0xd00b2c01
- maxSupply: 330000000
- halvingInterval: 515625
- initialReward: 33
- coinbaseMaturity: 20
- targetBlockTimeMs: 21000
- difficultyWindow: 120
- minDifficulty: 2
- maxDifficulty: 3
- addressFormat: dubz_ + sha256(pubkey).slice(0,12)

Node roles

- Bootstrap node — first trusted node other peers join from.
- Seed node — stable public-facing node used for discovery and sync.
- Miner node — mines blocks and relays transactions and blocks.
- Relay node — non-mining peer that syncs and relays network traffic.

- Local dev node — private/local testing node.

Required files

- wallet.miner.json
- dubzchain.peers.json
- rpc.send-history.json
- local chain data files
- snapshot/checkpoint files if enabled
- dist/ build output

Important endpoints

- /health — quick health check
- /status — summary node status
- /diagnostics — detailed runtime, chain, and network info
- /peers — connected peers
- /sync — sync progress
- /storage — storage/pruning mode
- /metrics — metrics output
- /debug/replay-verify — full replay consistency check
- /wallet/lookup — wallet lookup
- /wallet/validateSend — validate transaction inputs
- /wallet/buildTx — preview/build transaction
- /wallet/send — send transaction
- /index — explorer UI

Wallet flow

- Lookup wallet with /wallet/lookup or the explorer Wallet Lookup panel.
- Validate send inputs with /wallet/validateSend.
- Preview transaction with /wallet/buildTx.
- Send transaction with /wallet/send.
- Confirm in explorer send history, mempool, or block inclusion.

Sync behavior

- Peers connect from configured peers and the saved peer table.
- Headers are exchanged first.
- Missing blocks are fetched in ranges.
- Snapshot-assisted sync may speed up initial catch-up.

- Strongest chain by cumulative work wins.
- Fork comparisons can be checked with `/debug/fork-compare`.

Storage behavior

- Archival mode keeps full chain data.
- Pruned mode keeps checkpoint data plus a recent retained tail.
- Restart should preserve chain files and peer table.
- `/storage` can be used to confirm current storage mode.

Troubleshooting

- If RPC is down, verify the process is running and `rpcHost` is correct.
- If peers are zero, verify host, port, firewall, and advertise URL.
- If sync stalls, inspect `/sync`, `/peers`, and `/diagnostics`.
- If wallet lookup fails, verify the wallet file exists locally.
- If tx validation fails, review the returned error code.

Operator warnings

- Do not change `chainId`, `networkMagic`, `genesis`, `reward rules`, or `maturity` once public testnet is live.
- Do not mix old chain data with changed constants.
- Keep RPC private unless you intentionally expose it.
- Back up node data before upgrades.
- After restart or upgrade, verify `/health`, `/status`, and `/debug/replay-verify`.

Core startup commands

Local mining node: `node dist/index.js 3001 --host 127.0.0.1 --rpc-host 127.0.0.1 --automine --mine-empty`

Public bootstrap node: `node dist/index.js 3001 --host 0.0.0.0 --rpc-host 127.0.0.1 --advertise ws://YOUR_PUBLIC_IP:3001 --automine --mine-empty`

Public peer node: `node dist/index.js 3002 --host 0.0.0.0 --rpc-host 127.0.0.1 --peer ws://YOUR_BOOTSTRAP_IP:3001 --advertise ws://YOUR_PUBLIC_IP:3002`

Local secondary peer: `node dist/index.js 3002 --host 127.0.0.1 --rpc-host 127.0.0.1 --peer ws://127.0.0.1:3001`

6.2 Clean Launch Recipes

Recipe A — Single bootstrap node

Start a public bootstrap node with advertise URL and optional mining. Pass when explorer loads, /health returns ok, and blocks begin mining if automine is enabled.

Recipe B — Second node joins bootstrap

Start node 3002 pointed at the bootstrap peer. Pass when /peers shows the connection and the second node reaches the same tip.

Recipe C — Third node joins network

Start node 3003, point it at bootstrap, and confirm discovery expands the peer table. Pass when the node catches tip and stays connected.

Recipe D — Public seed node

Run a stable public node for discovery with a public P2P bind and private RPC. Pass when new peers can connect and sync from it.

Recipe E — Snapshot-assisted join

Join with a fresh node and let snapshot-assisted sync accelerate initial catch-up. Pass when the node continues normal sync after snapshot load and verifies state.

Recipe F — Restart / recovery

Stop and restart a node with the same command. Pass when chain files reload, peer table reloads, explorer returns, and replay/health checks pass.

Recipe G — Archival vs pruned

Run archival or pruned mode depending on your runtime flags. Pass when /storage reports the expected mode and restart behavior remains correct.

6.3 Seed-Node Deployment Guidance

- Use a stable Linux VPS or server with a reliable internet connection and enough disk for archival or retained pruned state.
- Expose the P2P port publicly and keep RPC bound to 127.0.0.1 unless you intentionally need public RPC.
- Use --host 0.0.0.0 for public P2P reachability and --advertise ws://PUBLIC_IP_OR_DOMAIN:PORT for discoverability.
- Persist wallet files, peer table files, chain data, and any snapshot/checkpoint data.
- Monitor /health, /status, /peers, /sync, /storage, /diagnostics, and /metrics.
- Before upgrades, back up chain files, confirm constants did not change, rebuild, restart, and verify replay plus peer health.

- If abuse appears, inspect peer and diagnostics data, keep RPC private, and review invalid-message handling counters/logs.

6.4 Frozen Network Constants

- chainId: dubzchain-devnet
- protocolVersion: 1
- networkMagic: 0xd00b2c01
- maxSupply: 33000000
- halvingInterval: 515625
- initialReward: 33
- coinbaseMaturity: 20
- targetBlockTimeMs: 21000
- difficultyWindow: 120
- minDifficulty: 2
- maxDifficulty: 3
- deterministic genesis required

Freeze rule: once public testnet is live, do not change chain ID, network magic, genesis, reward schedule, maturity, or difficulty rules without a planned reset or versioned transition.

6.5 Release Discipline

- Back up chain and peer files before every release.
- Release notes must clearly state whether network constants changed.
- If constants changed, treat the rollout as incompatible and plan a reset or explicit migration.
- After every release, verify /health, /status, /peers, /sync, and /debug/replay-verify.
- Keep one canonical bootstrap/seed node reference for public operators.
- Do not silently change public peer URLs or launch commands.

Phase 6 can be marked complete after you review placeholders such as YOUR_PUBLIC_IP/YOUR_BOOTSTRAP_IP and confirm any exact prune/archival runtime flags used in your current node build.